

PEARLS AQI Predictor

An End-to-End Serverless Air Quality Forecasting System for Karachi

Project Report

Prepared by

Syed Anas Azim

[Live dashboard](#)

[Github Repository](#)

Contents

1. Project Overview.....	3
2. System Architecture.....	3
3. Feature Pipeline	5
3.1 Data Source	5
3.2 Feature Engineering.....	5
3.3 Historical Backfill.....	6
4. Training Pipeline	6
4.1 Models and Evaluation Methodology.....	6
4.2 Benchmarking Against a Persistence Baseline.....	6
5. Automated CI/CD Pipeline	8
5.1 Scheduling	8
5.2 Handling Concurrent Workflows.....	8
6. Web Application / Dashboard.....	9
6.1 Header & Real-Time Overview.....	9
6.2 Environmental & Weather Context	9
6.3 Predictive Intelligence & Multi-Horizon Forecasting	10
6.4 SHAP Explainability	11
7. Explainability & Analytics.....	12
7.1 Exploratory Data Analysis	12
Distributions and outliers.....	12
Temporal patterns	13
Correlation and multicollinearity	15
Horizon-wise correlation decay	16
Autocorrelation, partial autocorrelation, and stationarity.....	17
7.2 SHAP Explainability	18
8. Difficulties Faced and Solution Used	20
1. Hopsworks Feature Store was slow and unreliable.....	20
2. GitHub Actions' built-in scheduler was unreliable.....	21
3. Two scheduled workflows writing to the same files caused race conditions.....	22
4. Partial or corrupted file reads.....	22
9. Critical Evaluation / Limitations	22
10. Conclusion & Future Work.....	23

1. Project Overview

Pearls AQI Predictor is a serverless system that forecasts Karachi's Air Quality Index (AQI) 24, 48, and 72 hours ahead. It has four parts: an hourly data pipeline that fetches weather and pollution data and stores it, a daily training pipeline that trains and compares multiple models, automated scheduling that keeps both running without manual work, and a Streamlit dashboard that shows the current AQI, the forecasts, and an explanation of what's driving each prediction.

Every hour, `update_data.py` pulls fresh weather and air-quality data from Open-Meteo, stores it in the Hopsworks Feature Store, and appends it to a local CSV file. Once that finishes, `inference.py` runs automatically, loads the best model for each horizon from the Hopsworks Model Registry, generates the 24h/48h/72h predictions, computes SHAP values to explain them, and saves the results. Once a day, `model_training.py` reads the full historical data, retrains three models — Ridge Regression, Random Forest, and a Neural Network — for each horizon, evaluates them, and registers all three in the Model Registry while marking the best one per horizon as the one to use. The dashboard (`app.py`) reads the latest cached data and displays it.

2. System Architecture

The system is organised as four loosely-coupled jobs — hourly ingestion, hourly inference, daily training, and the dashboard — connected by a small set of shared artifacts: two Hopsworks-hosted stores (Feature Store and Model Registry) and three CSV files committed to the repository that act as a fast, always-available cache. Everything runs on GitHub Actions; there is no persistent server other than the Streamlit Cloud instance hosting the dashboard.

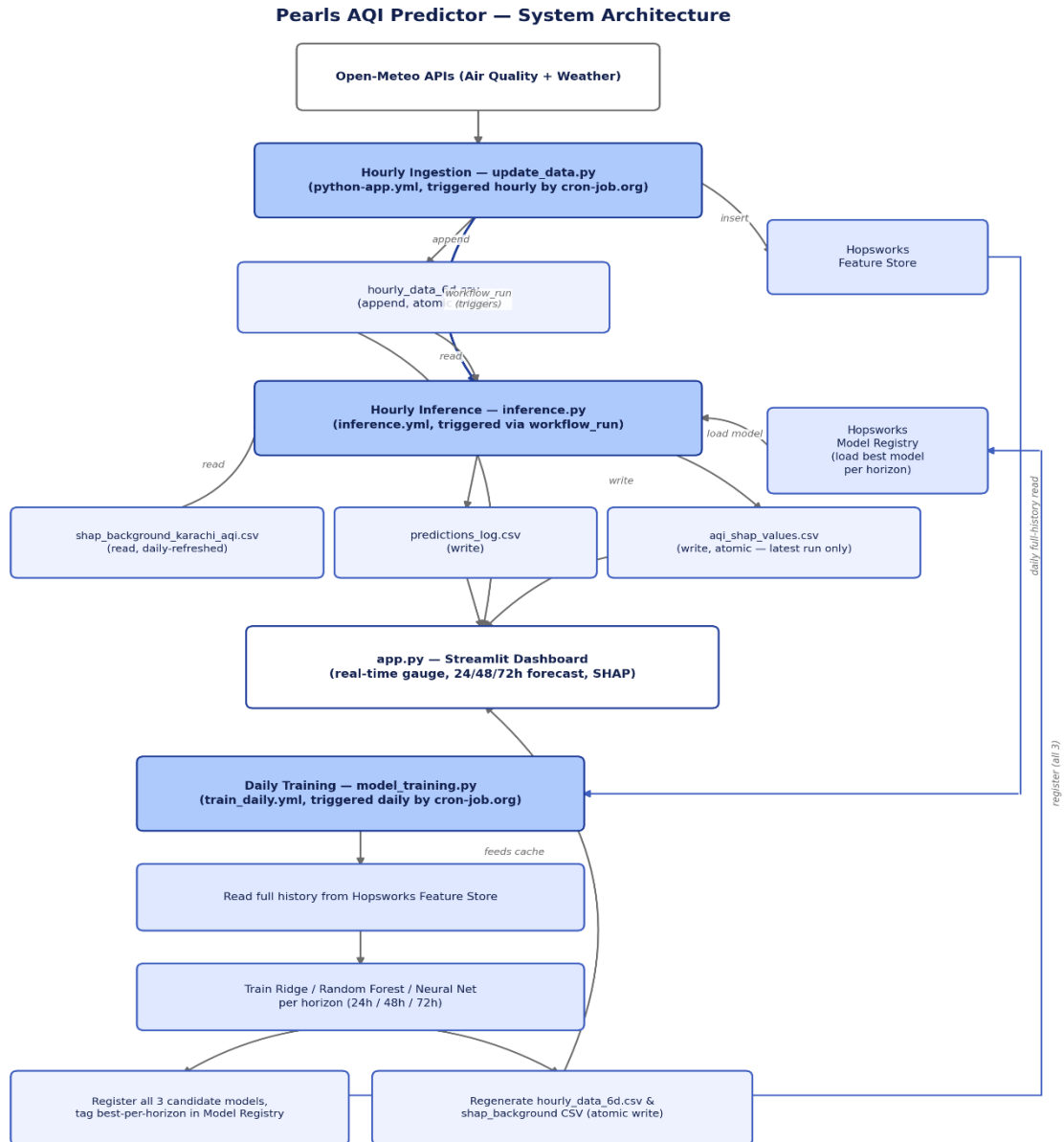


Figure 1. End-to-end system architecture: hourly ingestion, hourly inference, daily training, and the dashboard, connected through Hopsworks and repository-committed CSV caches.

Data flows in two independent cycles that share the same underlying stores. On the hourly cycle, `update_data.py` pulls the latest weather and air-quality readings from Open-Meteo, inserts them into the

Hopsworks Feature Store, and appends the new row to `hourly_data_6d.csv`, a rolling roughly six-day window kept in the repository. Completion of that workflow triggers `inference.py`, which reads the six-day CSV window (not the Feature Store) to build the current feature row, loads the best-performing model per horizon from the Model Registry, produces 24h/48h/72h predictions, computes SHAP values against a pre-built background sample, and writes both `predictions_log.csv` and `aqi_shap_values.csv`.

On the daily cycle, `model_training.py` performs the one Feature Store read of the day — a full-history pull — trains all three candidate models per horizon, registers all of them in the Model Registry while tagging the best one per horizon, and regenerates both `hourly_data_6d.csv` and the SHAP background sample from scratch so they stay consistent with the latest data and models.

The dashboard (`app.py`) never talks to Hopsworks directly at all: it reads only the three CSV files, which is what keeps page loads fast regardless of how Hopsworks is behaving at that moment. The key architectural principle is a deliberately narrow role for Hopsworks: the Feature Store is touched only during the hourly insert and the once-daily full-history read; the Model Registry, by contrast, is still called every hour during inference to fetch the current best model, which in practice was never a performance problem. This split — and why it exists — is explained in full in Section 7.1.

3. Feature Pipeline

3.1 Data Source

The system uses [Open-Meteo's](#) Air Quality and Weather APIs rather than AQICN or OpenWeather. The project specification explicitly allows exploring alternative data sources, and Open-Meteo was chosen for its free tier, hourly historical archive, and combined access to both pollutant concentrations and meteorological variables from a single provider.

3.2 Feature Engineering

All feature computation lives in `scripts/feature_engineering.py`, which is imported by both `model_training.py` and `inference.py`. This was a deliberate design choice: because both the training and serving code paths call the exact same functions (`clean_raw`, `compute_features`, `add_targets`, `get_feature_sets`, plus the shared constants `HORIZONS`, `LAGS`, `HORIZON_TO_LEAD_DAYS`, and `HORIZON_TO_HOURS`).

The feature set, informed by the exploratory analysis, includes:

- Raw pollutant and weather readings: PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone, dust, temperature, relative humidity, wind speed and direction, surface pressure, precipitation, and cloud cover.

- Short-term autoregressive lags of AQI at 1, 2, and 3 hours — selected because PACF analysis showed these as the only lags with a strong, non-redundant direct effect.
- Daily-seasonality lags of AQI at 24, 48, and 72 hours, chosen because they align both with a secondary PACF spike cluster and with the three forecast horizons themselves.
- 24-hour rolling mean and standard deviation of AQI, capturing whether the current period is a stable or volatile stretch.
- Cyclic (sine/cosine) encodings of hour-of-day, day-of-week, month, and wind direction, so that adjacent values on a cycle (e.g. hour 23 and hour 0) are treated as close rather than far apart.
- Target columns for AQI at $t+24h$, $t+48h$, and $t+72h$, produced by shifting the AQI series forward.

3.3 Historical Backfill

A one-time, unscheduled script, `scripts/historical_data_script.py`, was used to backfill the Feature Store with historical hourly data before the automated hourly pipeline took over. This satisfies the specification's backfill requirement and is intentionally separate from the recurring hourly/daily jobs.

4. Training Pipeline

4.1 Models and Evaluation Methodology

Three model families are trained independently for each of the three forecast horizons (24h, 48h, 72h), spanning the range the specification asks for — statistical, tree-ensemble, and deep learning:

- Ridge Regression — a regularised linear model, tuned over a range of alpha values on the validation split.
- Random Forest — an ensemble of regression trees, tuned for depth and minimum leaf size to control overfitting.
- A feed-forward Neural Network (Keras/TensorFlow MLP) with dropout and early stopping.

Every model is evaluated with the three metrics the specification calls for — Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 — computed on a held-out test split that follows the training and validation splits chronologically, so no future data leaks into training.

4.2 Benchmarking Against a Persistence Baseline

Raw error metrics are easy to misread in isolation, so every model was additionally compared against a naive persistence baseline: predicting that AQI at $t+h$ will simply equal AQI right now. A model that cannot beat persistence has no real forecasting value, regardless of how its R^2 or RMSE looks on its own. The comparison below is taken from the exploratory model-training notebook (using a data snapshot through 22 August 2026, before the production pipeline's daily retraining took over) and is illustrative of relative model behaviour rather than the exact numbers served by any single day's production model.

Horizon	Model	Test MAE	Test RMSE	Test R^2	RMSE vs. Persistence
24h	Persistence (baseline)	6.432	9.352	0.335	—
24h	Ridge Regression	5.415	7.830	0.534	-16.3%
24h	Random Forest	5.412	7.743	0.544	-17.2%
24h	Neural Network	5.307	7.803	0.537	-16.6%
48h	Persistence (baseline)	9.287	13.123	-0.287	—
48h	Ridge Regression	7.646	10.823	0.124	-17.5%
48h	Random Forest	7.395	10.641	0.154	-18.9%
48h	Neural Network	7.524	11.133	0.073	-15.2%
72h	Persistence (baseline)	10.672	15.260	-0.733	—
72h	Ridge Regression	8.284	11.755	-0.028	-23.0%
72h	Random Forest	7.462	11.306	0.049	-25.9%
72h	Neural Network	8.137	12.280	-0.122	-19.5%

Table 1. Test-set MAE, RMSE, and R^2 for each candidate model at each horizon, with RMSE improvement over the persistence baseline (negative = lower error than persistence, i.e. better).

Every model beats persistence on RMSE at every horizon, by 15–26%, which is the strongest evidence that the pipeline is learning genuine signal rather than an artefact of the metrics. R^2 tells a more cautionary story: it degrades sharply with horizon for every model, and both Ridge and the Neural Network turn negative at 72h, meaning they explain less of that period's AQI variance than simply using its own mean would. Random Forest is the only model that stays positive at every horizon and shows the largest relative improvement over persistence at 72h, which is also where the forecasting problem is hardest. This, combined with Random Forest's native support for feature importances and SHAP, made it the strongest overall candidate in the exploratory notebook, and it also wins the production selection on most days.

Two limitations of this comparison are worth being explicit about. First, the validation and test splits in the exploratory notebook cover different, non-overlapping seasonal windows (roughly January–April and April–August respectively), so month-based features may partly be acting as a proxy for split membership rather than pure seasonality — this is flagged again alongside the SHAP findings it affects. Second, test error was lower than validation error for every model at every horizon, suggesting the test period happened to contain less volatile AQI than the validation period, rather than the models generalising unusually well; R^2 , being normalised against each period's own variance, is the more trustworthy of the two error signals here.

5. Automated CI/CD Pipeline

5.1 Scheduling

The pipeline uses three GitHub Actions workflows: `python-app.yml` (hourly ingestion), `train_daily.yml` (daily training), and `inference.yml` (hourly inference).

The hourly and daily jobs are triggered by an external scheduler, `cron-job.org`, which calls each workflow over HTTP at the set time, instead of using GitHub Actions' own built-in schedule: `trigger`. This is because GitHub's native scheduler wasn't firing on time during testing — it would sometimes run a couple of hours late or skip ahead with 2–3 hour gaps between runs. `cron-job.org` fires consistently on schedule, so it's used for both the hourly ingestion and daily training jobs.

`inference.yml` doesn't need its own schedule. It's set up to trigger automatically right after `python-app.yml` finishes, using GitHub's `workflow_run` event. This is done because inference should always run right after new data comes in, not on a fixed clock of its own.

5.2 Handling Concurrent Workflows

Since the hourly and daily workflows both write to the same CSV files in the repo, a few issues came up around running them at the same time, and were fixed along the way:

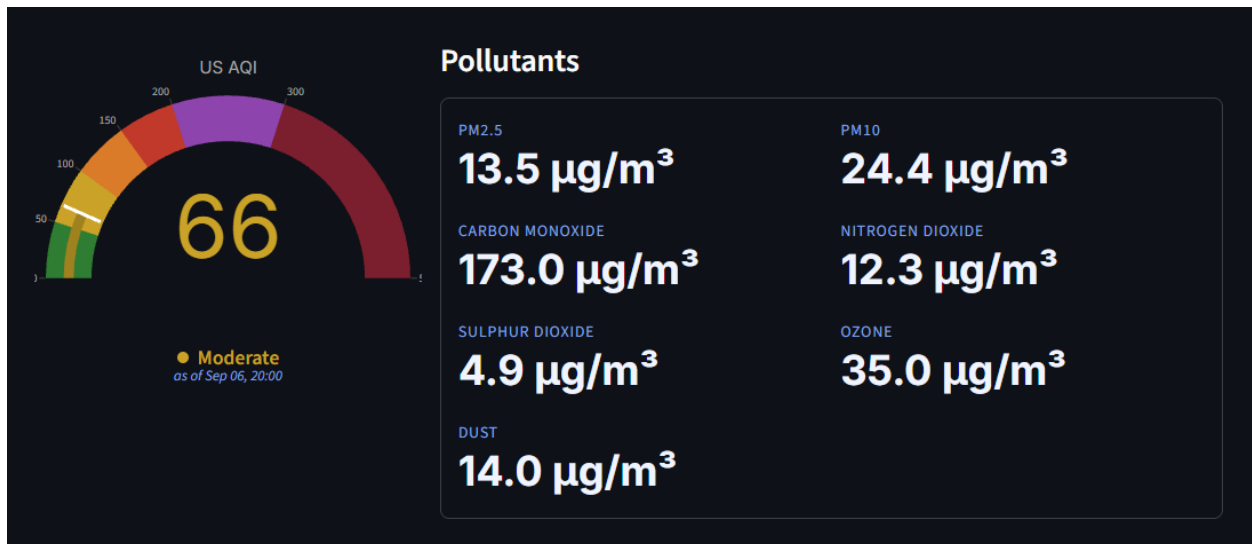
- Both workflows writing to `hourly_data_6d.csv` at the same time occasionally caused push conflicts. This was fixed by adding a shared concurrency group in GitHub Actions so the two jobs can't push to that file simultaneously.
- Running `git pull --rebase` after the script had already changed local files caused a "cannot pull with rebase: unstaged changes" error. Fixed by reordering the steps: commit the local changes first, then pull with rebase, then push.
- When both jobs did end up rebasing against each other, git couldn't merge two completely regenerated versions of the same CSV automatically. This was resolved using `git pull --rebase -X theirs`, since these files are fully regenerated by the scripts each time anyway — there's nothing to manually merge, and the next run will produce a correct file regardless.
- To avoid a script reading a CSV while another process is still writing to it, every CSV write in the project (ingestion, training, and inference) uses atomic writes: write to a temp file first, then swap it into place, so a reader never sees a half-written file.
- A few bugs were caught during review and fixed: a missing import time that would have crashed the retry logic, a data-fetch function that was accidentally being called twice in one run, and a SHAP-logging step that was overwriting the previous horizon's row instead of keeping all three.
- `inference.yml` also has its own concurrency group, so if one run takes unusually long, the next hourly trigger won't start a second overlapping run against the same files.

6. Web Application / Dashboard

- The dashboard is a Streamlit application (app.py) that gives a single view of Karachi's current air quality, the 24/48/72-hour forecasts, and the reasoning behind them. It's built with a dark-themed layout designed to keep a fairly complex set of ML outputs easy to read at a glance, and it reads only from the cached CSV files described in Section 3, so it loads quickly regardless of anything happening on the backend.

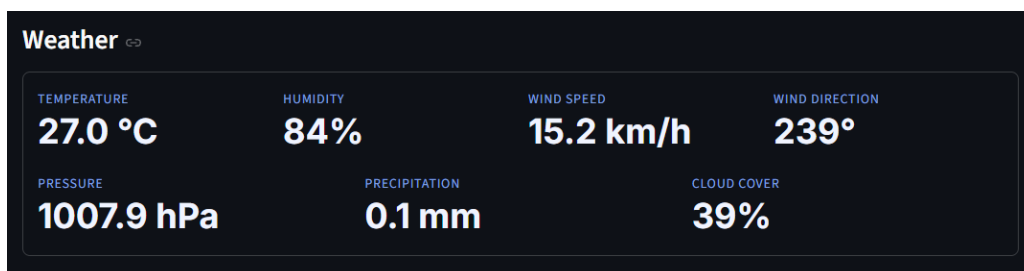
6.1 Header & Real-Time Overview

- The top of the dashboard shows the location being tracked — Karachi, Pakistan (24.8607°N, 67.0011°E) — along with a note on how the forecast is generated (the daily-selected best model among Ridge, Random Forest, and Neural Network, per horizon). The current US AQI is shown as a large color-coded radial gauge for instant visual read (e.g. AQI 67, "Moderate"), alongside a breakdown of the current concentrations of the key pollutants — PM2.5, PM10, CO, NO₂, SO₂, O₃, and dust — each in µg/m³.



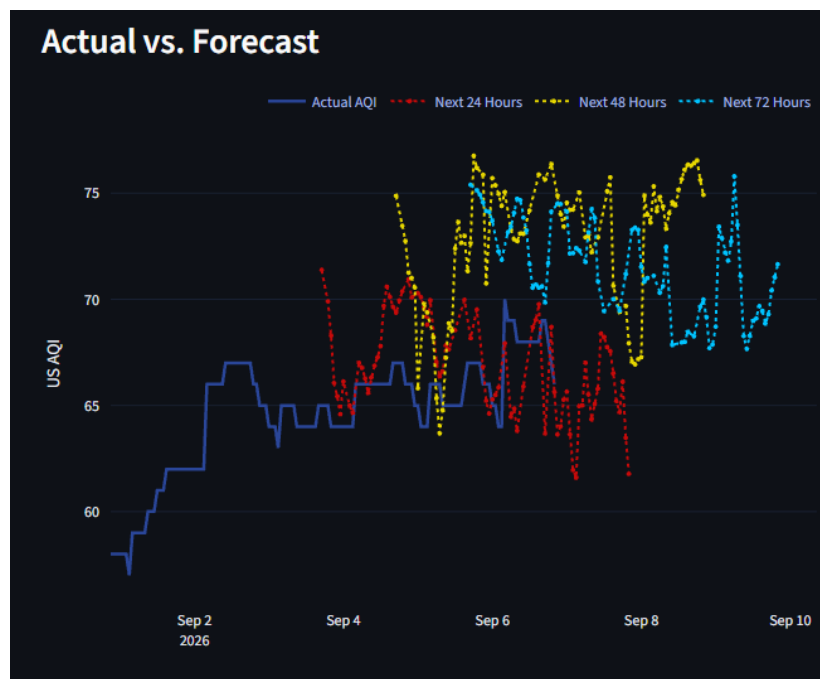
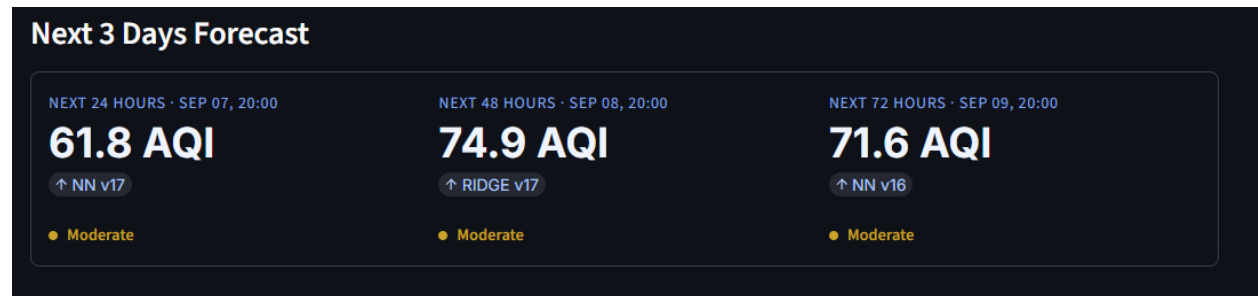
6.2 Environmental & Weather Context

- Below that is a summary of the current weather conditions that influence how pollution disperses: temperature (°C), humidity (%), wind speed (km/h) and direction (°), surface pressure (hPa), precipitation (mm), and cloud cover (%).



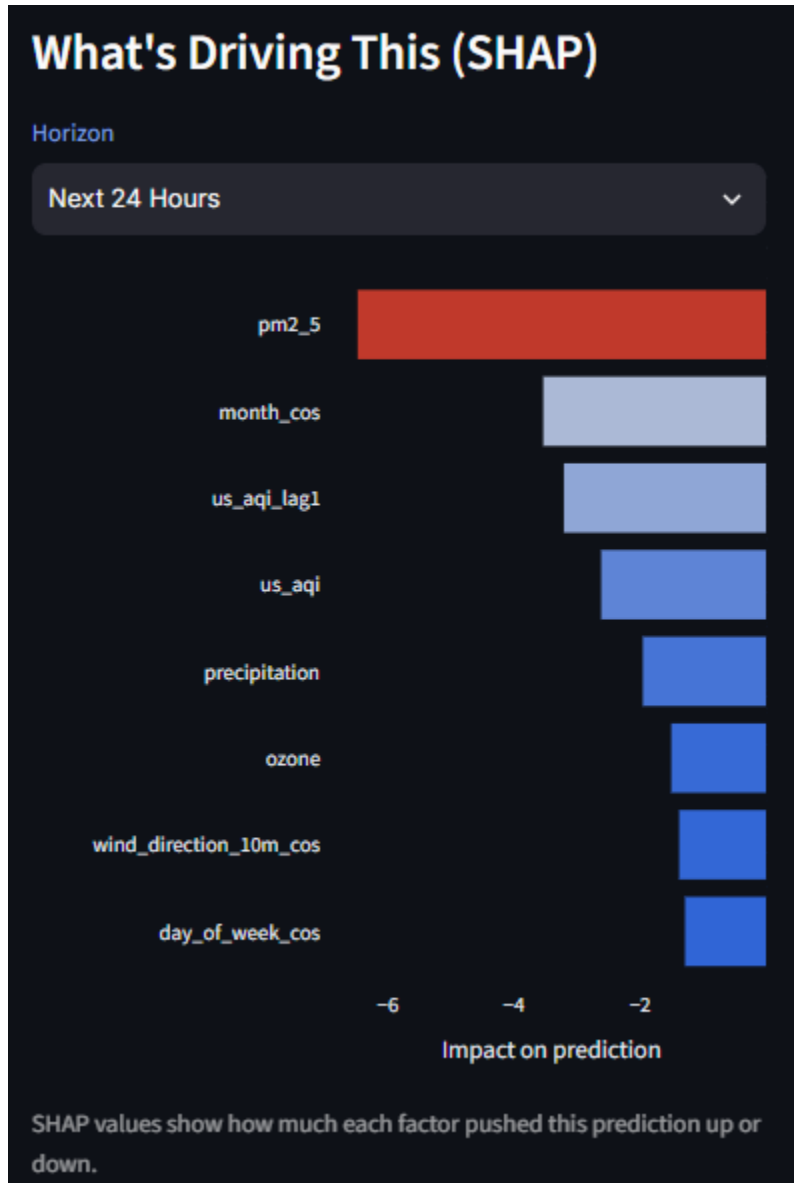
6.3 Predictive Intelligence & Multi-Horizon Forecasting

- The forecast section shows the predicted AQI for the next 24, 48, and 72 hours, each as its own card. Each card also shows which model produced that forecast (e.g. "NN v17" or "Ridge v17"), since a different model can be selected for each horizon on any given day, along with the resulting health category (Good, Moderate, Unhealthy, etc.).
- Below the forecast cards is a time-series trend chart plotting actual historical AQI against the forecasted values for all three horizons (24h, 48h, and 72h) on the same timeline, so it's possible to visually compare how closely each horizon's predictions have tracked the real AQI over time.



6.4 SHAP Explainability

- A SHAP feature-importance chart explains the forecast for the selected horizon, showing which factors pushed the prediction up or down and by how much. Two collapsible sections, "About this forecast" and "Model performance," sit at the bottom of the page for anyone who wants more detail, without cluttering the main view.



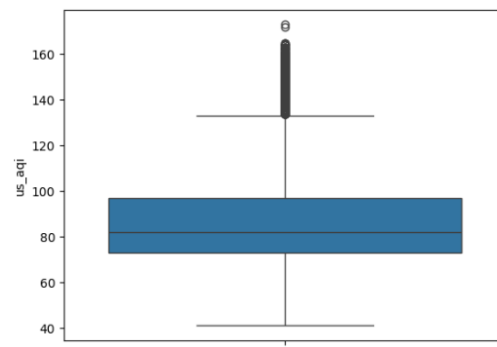
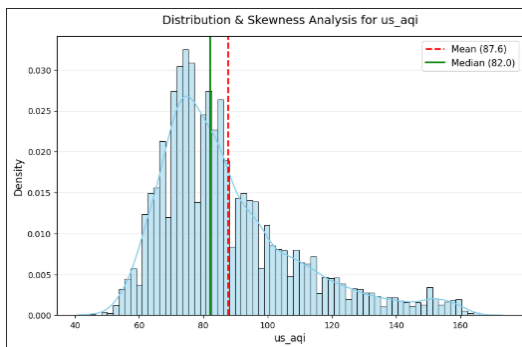
7. Explainability & Analytics

7.1 Exploratory Data Analysis

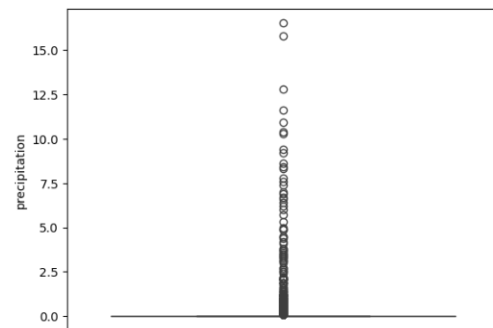
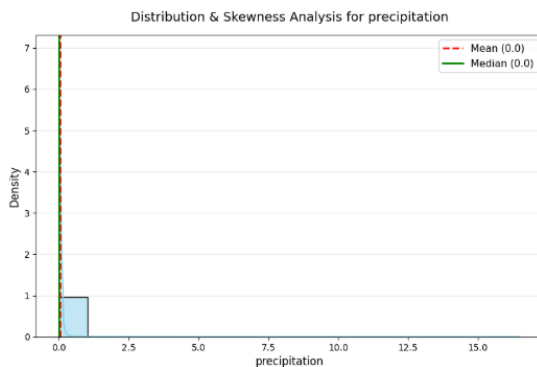
EDA was carried out on a roughly two-year snapshot of hourly Karachi data (August 2024 – August 2026), after cleaning: no duplicate rows were found, and only a small number of individual missing hours (never a run of consecutive hours) were present, caused by the automated pipeline occasionally recording a new row slightly late; these were filled with time-based interpolation rather than dropped.

Distributions and outliers

- Most pollutant columns (PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, dust) are right-skewed, consistent with long stretches of moderate pollution punctuated by occasional sharp spikes; outlier rates ranged from roughly 1.7% (ozone, the best-behaved pollutant) to 7.1% (carbon monoxide).
- us_aqi itself, the forecast target, is right-skewed (skew = 1.21) with genuine high-AQI outliers on the upper side — these represent real severe-pollution episodes and were deliberately kept rather than removed, since capturing them is the point of the forecasting task.



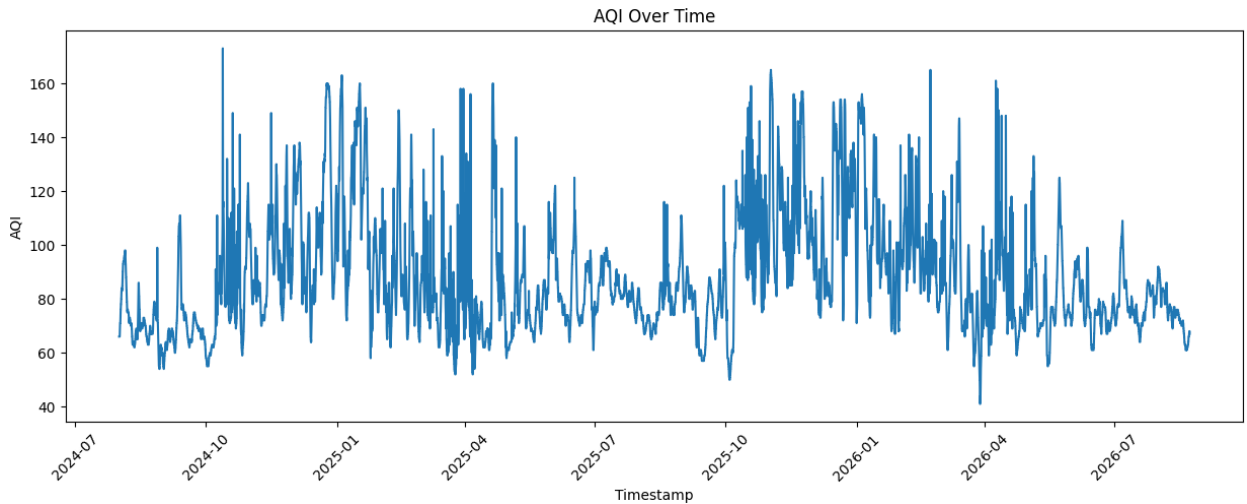
- Precipitation is by far the most skewed column (skew = 24.4), reflecting Karachi's long dry spells punctuated by rare rain events; surface pressure and relative humidity were the most symmetric, well-behaved columns in the dataset.



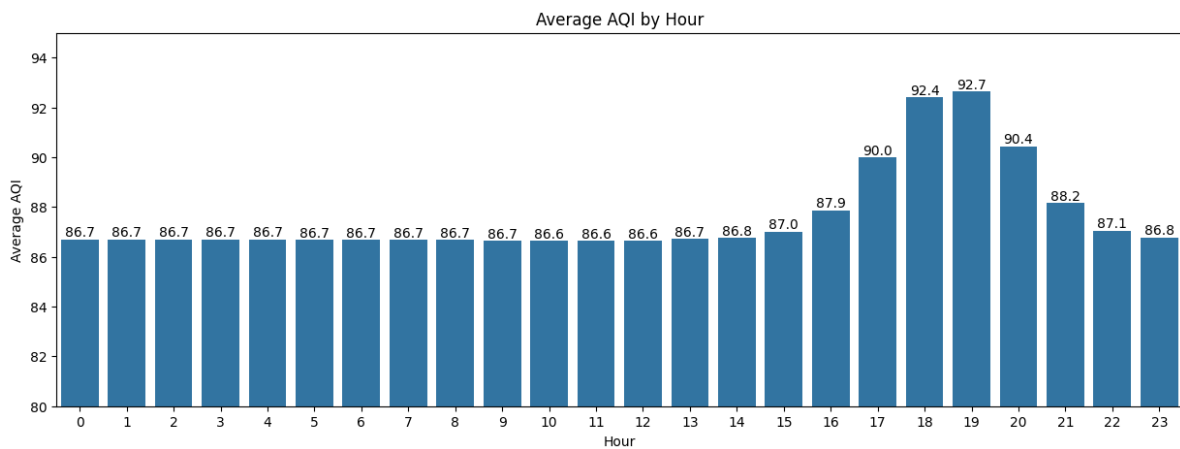
- Wind direction is a circular variable, so its raw skew/outlier statistics are not meaningful in the usual sense; it was cyclic-encoded (sin/cos) rather than relied upon as-is.

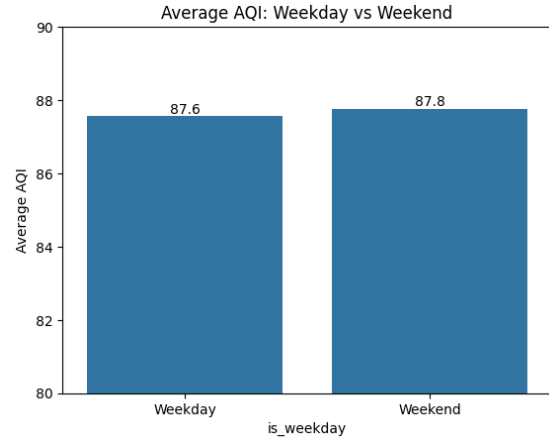
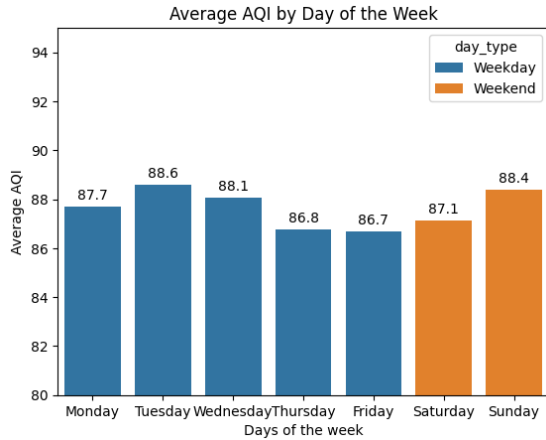
Temporal patterns

- AQI ranged from 41 to 173 across the dataset (mean ≈ 87.6), showing sustained multi-day pollution episodes rather than pure hour-to-hour noise — consistent with the strong short-term autocorrelation found in the ACF/PACF analysis below.

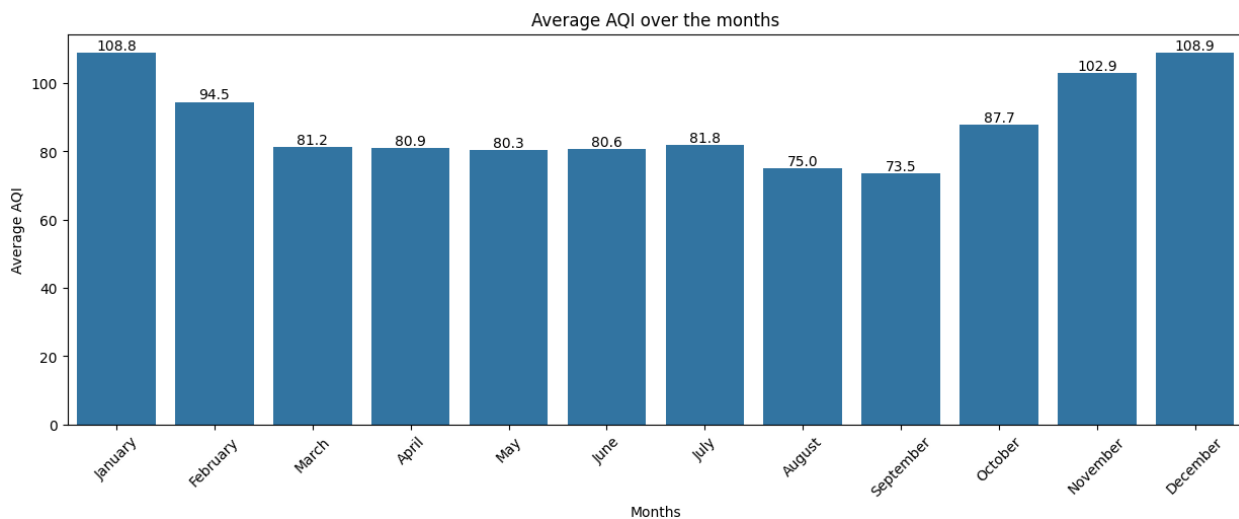


- Hour-of-day and day-of-week effects are both weak: AQI is fairly flat through the day with a modest evening peak around hour 19 (~ 92.7 vs. a trough of ~ 86.6 at hour 10), and weekday vs. weekend averages differ by under a point — suggesting traffic/weekday activity is not a dominant driver of AQI in this dataset.



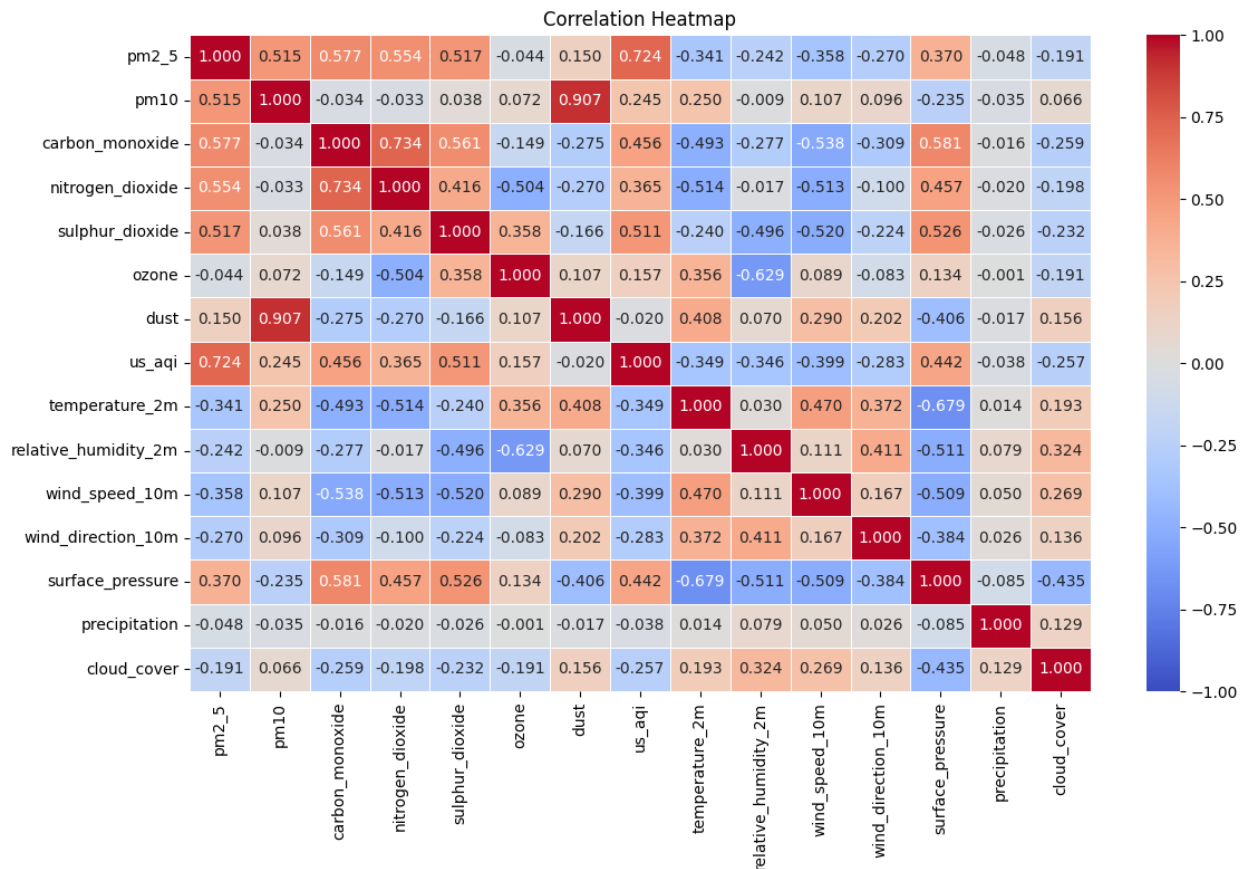


- Seasonality, by contrast, is the strongest temporal pattern found: winter months are far more polluted than summer, with December (108.9), January (108.8), and November (102.9) the worst, and September (73.5) and August (75.0) the best — a roughly 35-point swing consistent with well-documented winter smog/temperature-inversion effects in South Asian cities.



Correlation and multicollinearity

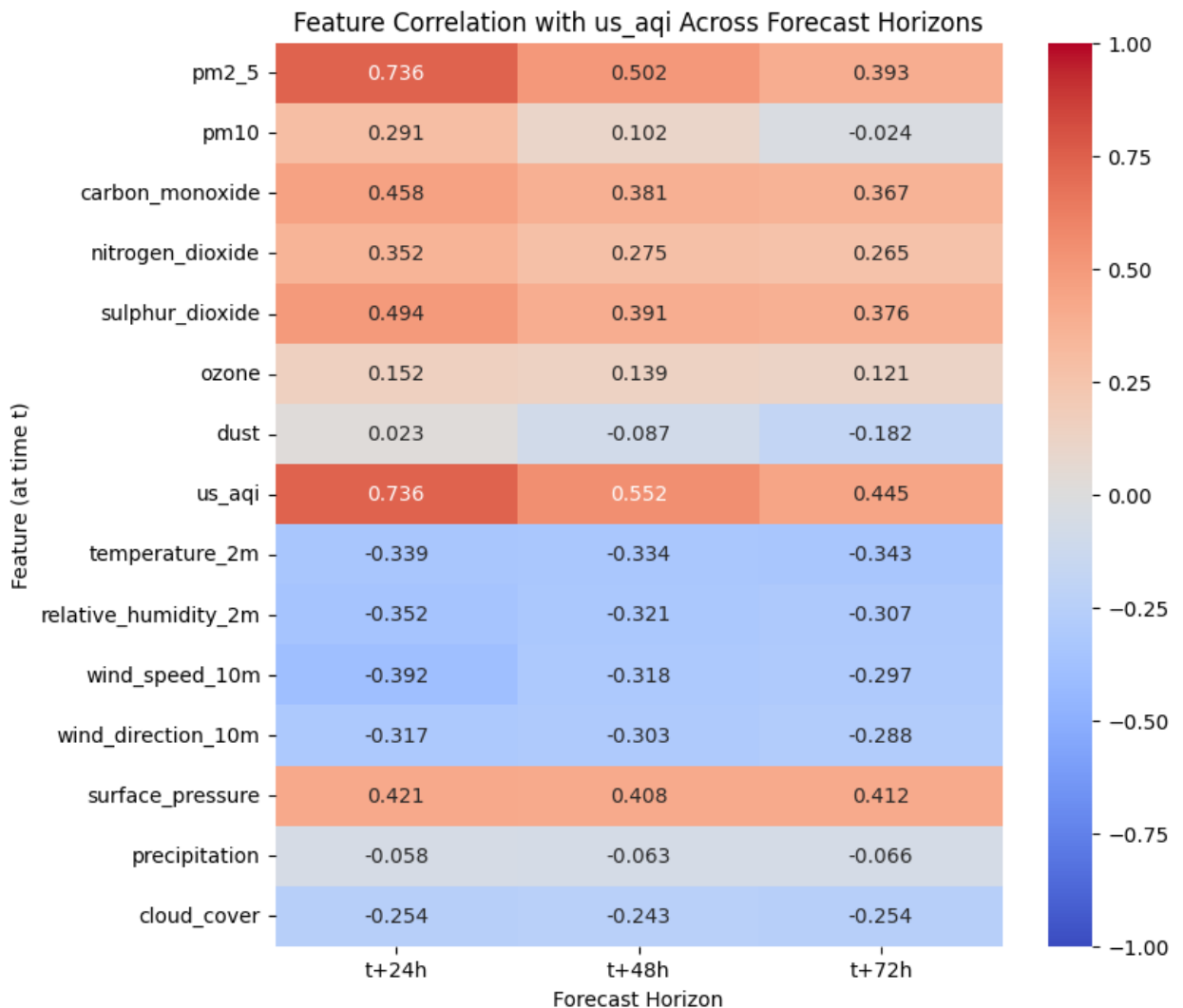
- PM2.5 is the dominant driver of AQI ($r = 0.724$), with sulphur dioxide ($r = 0.511$), carbon monoxide ($r = 0.456$), and surface pressure ($r = 0.442$) as moderate secondary drivers; ozone is essentially uncorrelated with AQI ($r = -0.020$).
- Multicollinearity was checked ahead of using linear models: PM10 and dust are near-duplicates ($r = 0.907$), and carbon monoxide and nitrogen dioxide are strongly correlated ($r = 0.734$), both consistent with shared traffic/combustion sources. Dust was dropped for linear/regularised and neural-network models to avoid this, but retained for tree-based models, which are not harmed by collinear inputs.



- A pair-plot analysis showed the PM2.5–AQI relationship is nonlinear (concave, rising steeply then plateauing) rather than a straight line — consistent with AQI's own piecewise-breakpoint calculation — and that AQI itself is bimodal (a dominant cluster around 70–90, plus a distinct smaller cluster near 150–160 representing severe-pollution episodes). Both observations motivated testing nonlinear models rather than relying on Ridge alone.

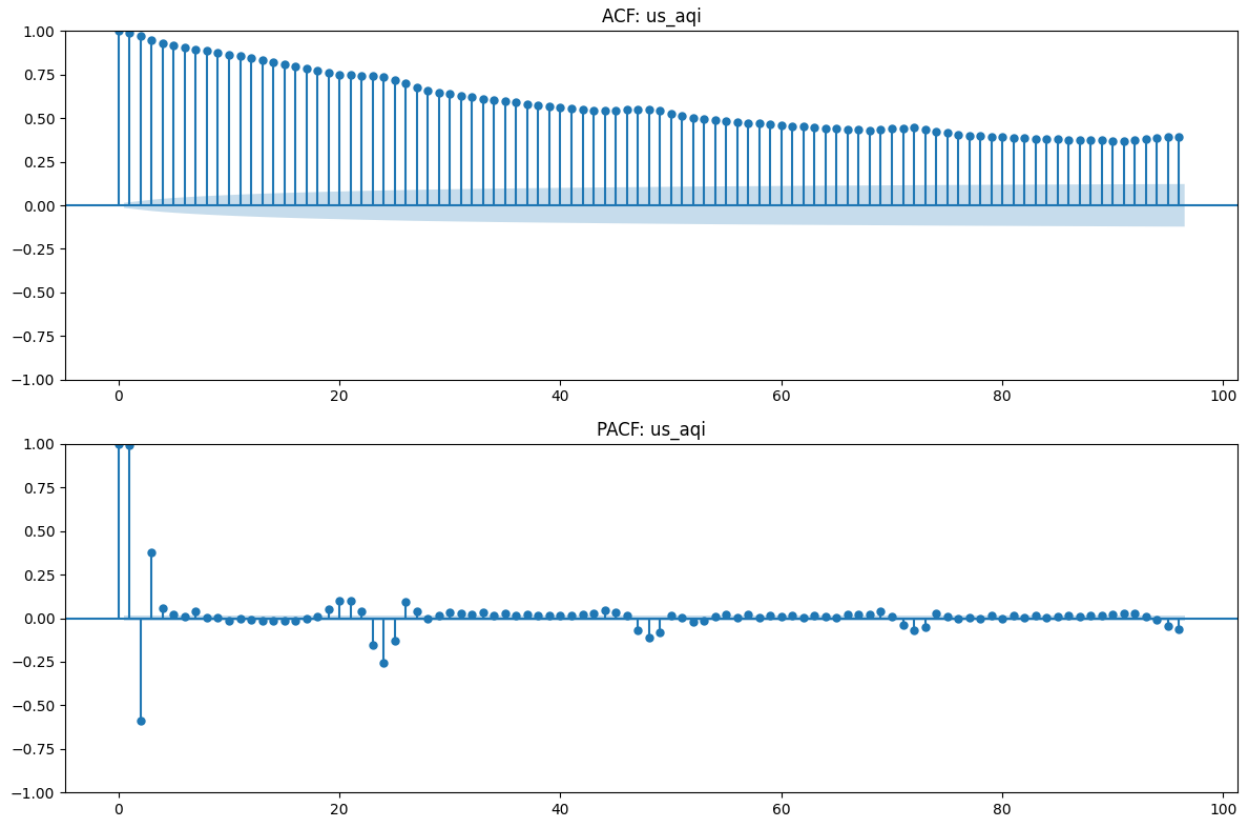
Horizon-wise correlation decay

- Every feature's correlation with AQI decays as the forecast horizon grows: AQI's own autocorrelation drops from 1.00 at t+0 to 0.445 at t+72h, and PM2.5 (the strongest external predictor at every horizon) decays from $r = 0.724$ to $r = 0.393$ over the same range.
- Surface pressure is the most horizon-robust feature, decaying only ~7% by t+72h and remaining a moderately strong predictor even three days out, likely because pressure systems evolve over multi-day timescales.
- This decay pattern directly motivated per-horizon models rather than a single shared model: short-horizon forecasts should lean on PM2.5 and recent AQI lags, while long-horizon forecasts should upweight surface pressure and gas pollutants, which retain relatively more signal.



Autocorrelation, partial autocorrelation, and stationarity

- The ACF for AQI decays very slowly (still ~ 0.4 at a 96-hour lag), indicating strong long-memory behaviour, while the PACF shows a clear short-term structure at lags 1–3 plus a secondary cluster around lag 24 (and faint echoes at 48 and 72) — directly informing the choice of lag features described in Section 4.2.



- An Augmented Dickey-Fuller test rejected the null hypothesis of a unit root (statistic = -7.578 , $p < 0.0001$), confirming the AQI series is statistically stationary despite its slow-decaying ACF. These two facts are not contradictory: the series has no long-term trend, while still carrying strong short-to-medium-term memory. Practically, this meant lag features could be built directly on raw AQI without differencing.

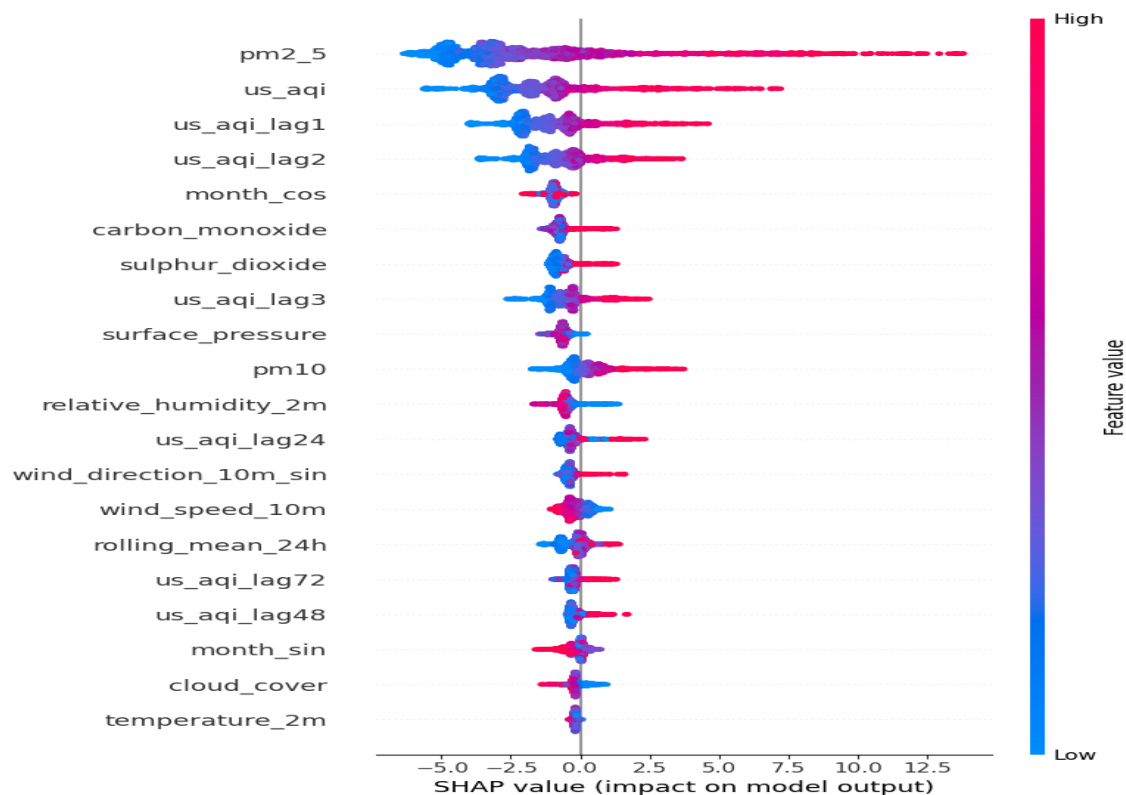
7.2 SHAP Explainability

SHAP was used with an explainer chosen per model type: `shap.TreeExplainer` for Random Forest, `shap.LinearExplainer` for Ridge, and `shap.KernelExplainer` for the neural network. In the production system, the `aqi_shap_values.csv` file is fully overwritten on every hourly inference run rather than appended to — a deliberate choice, since the dashboard's SHAP section is meant to answer “why does this hour's forecast look this way,” not to provide an aggregate or historical feature-importance view across many runs.

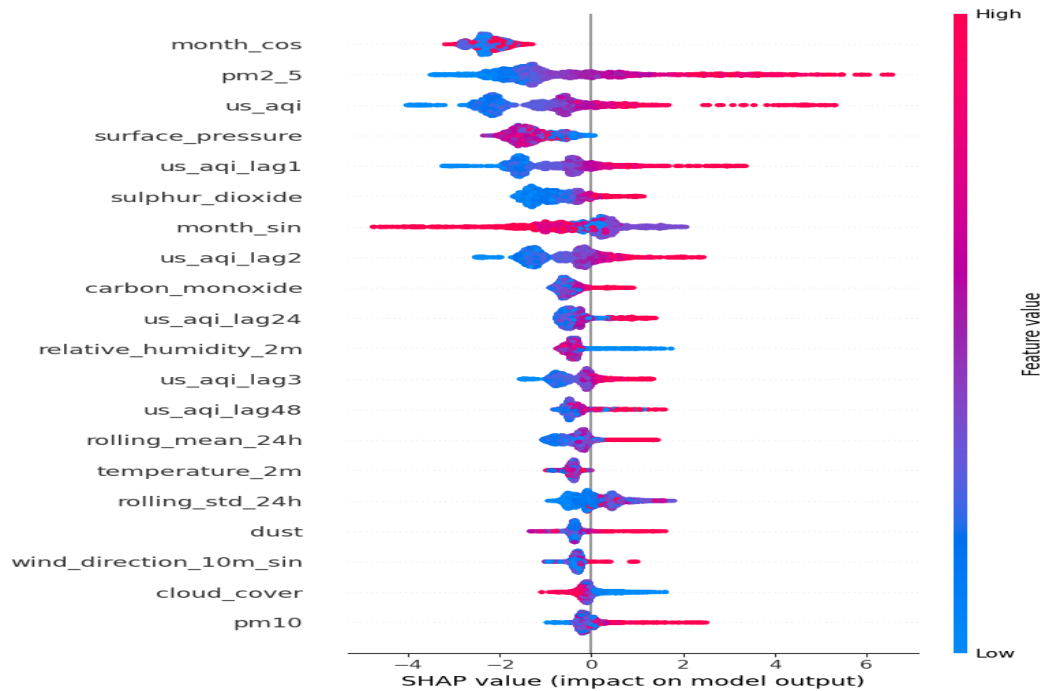
The SHAP background sample (`shap_background_karachi_aqi.csv`) is a stratified sample across (month, hour) buckets, refreshed daily during training, chosen specifically to avoid the autocorrelation and seasonal-coverage problems that would come from using a raw recent-hours window as the background instead.

In the exploratory notebook, SHAP was run on Random Forest, the strongest candidate, with findings that reinforce the correlation and feature-importance analysis above:

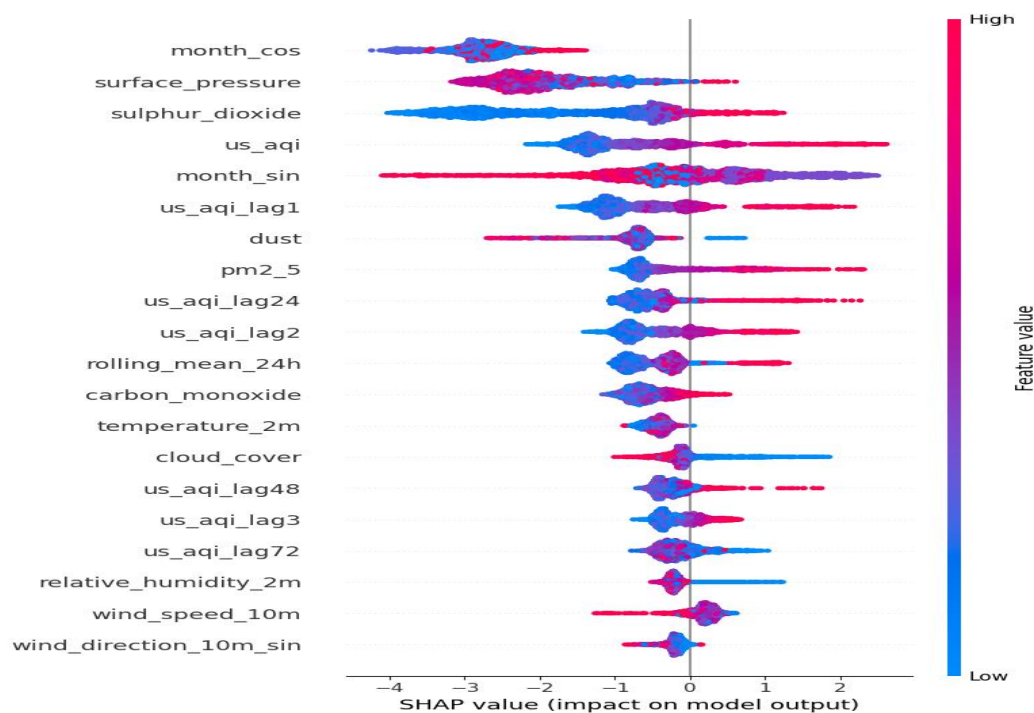
- At t+24h, PM2.5 dominates SHAP importance, followed by AQI and its short lags — consistent with the correlation and feature-importance findings, and with an intuitive direction (higher PM2.5 pushes the predicted AQI up).



- At t+48h, month_cos rises to the top feature, with PM2.5 and AQI lags still relevant but less dominant.



- At t+72h, month_cos and month_sin are the top two features — ahead of AQI, surface pressure, and sulphur dioxide — while PM2.5 falls to eighth place, a striking reversal from t+24h.



As flagged, this rising importance of month at longer horizons should be treated with some caution: because the validation and test splits used in the exploratory notebook cover only partial, non-overlapping seasonal windows, part of month's apparent importance may reflect the model using it as a proxy for split membership rather than a purely causal seasonal effect. This is a limitation of the current train/validation/test split rather than of Random Forest or SHAP themselves, and is called out again.

8. Difficulties Faced and Solution Used

A few real problems came up while building the pipeline, and each one led directly to a design decision in the final system.

1. Hopsworks Feature Store was slow and unreliable

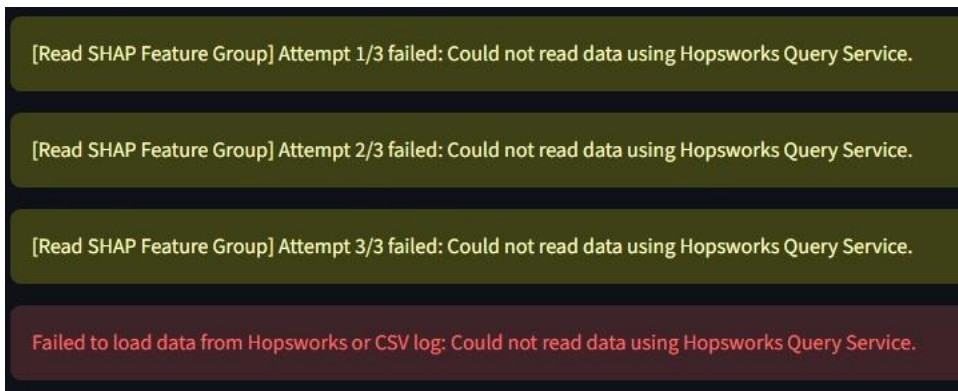
Problem: Reading from the Hopsworks Feature Store (`fg.read()`) was inconsistent — sometimes it returned in under a minute, sometimes it took over 15 minutes, with no clear pattern to predict which. On top of that, reads frequently failed outright with the error:

"Could not read data using Hopsworks Query Service."

Writes had a separate problem — concurrent inserts to the same feature group occasionally caused:

```
CommitFailedError: Failed to commit transaction: Commit failed: a concurrent transactions added new data
```

There was also a `ClusterJDatastoreException` ("Tuple did not exist", error code 626) during model registration, most likely caused by two training runs hitting Hopsworks' backend metadata store at the same time.



1Dashboard error when loading data from HOPSWORKS feature store

Solution: Reading from the Feature Store on every hourly inference run and every dashboard load wasn't reliable enough, so its use was scoped down to just two jobs: inserting new data every hour, and one full-history read per day during training. Everywhere else — hourly inference, the SHAP background sample, and the dashboard — now reads from CSV files that get refreshed on the same schedule instead. This brought the inference workflow's runtime down from an unpredictable range to a consistent few minutes, and removed the random read failures entirely. The Model Registry never had this problem, so it's still called normally every hour to load the current best model.

✖ Run Inference Run Inference #71: completed by SyedAnasAzim	📅 Sep 5, 11:11 GMT+5 🕒 2h 4m 10s ...
✔ Update Data Update Data #214: Manually run by SyedAnasAzim main	📅 Sep 5, 11:00 GMT+5 🕒 10m 44s ...
✔ Run Inference Run Inference #70: completed by SyedAnasAzim	📅 Sep 5, 10:12 GMT+5 🕒 1h 37m 10s ...
✔ Update Data Update Data #213: Manually run by SyedAnasAzim main	📅 Sep 5, 10:00 GMT+5 🕒 11m 59s ...
✖ Run Inference Run Inference #69: completed by SyedAnasAzim	📅 Sep 5, 09:08 GMT+5 🕒 1h 4m 15s ...

2Runtime of Run inference when getting the features from HOPSWORKS feature store

✔ Run Inference Run Inference #105: completed by SyedAnasAzim	📅 3 minutes ago 🕒 2m 46s ...
✔ Update Data Update Data #248: Manually run by SyedAnasAzim main	📅 15 minutes ago 🕒 11m 52s ...
✔ Run Inference Run Inference #104: completed by SyedAnasAzim	📅 Today at 20:14 🕒 2m 44s ...
✔ Update Data Update Data #247: Manually run by SyedAnasAzim main	📅 Today at 20:00 🕒 14m 2s ...
✔ Run Inference Run Inference #103: completed by SyedAnasAzim	📅 Today at 19:22 🕒 2m 35s ...

3Runtime of Run Inference when using csv cached hot path

2. GitHub Actions' built-in scheduler was unreliable

Problem: GitHub Actions has a native schedule: trigger for running workflows on a timer, but during development it fired inconsistently — sometimes a couple of hours late, sometimes with 2–3 hour gaps between runs instead of firing on the hour as expected. For a pipeline where hourly data and daily retraining need to happen on time, this wasn't good enough.

Solution: The hourly ingestion and daily training workflows were switched to an external scheduler, cron-job.org, which calls each GitHub Actions workflow over HTTP at the correct time. This fixed the timing issue completely. The one workflow that didn't need this fix is inference.yml, since it's triggered directly after the hourly ingestion job finishes, using GitHub's workflow_run event, rather than running on its own clock.

3. Two scheduled workflows writing to the same files caused race conditions

Problem: Since the hourly and daily workflows both write to the same CSV files in the repository, a few conflicts came up when they ran close together: push conflicts on hourly_data_6d.csv, a "cannot pull with rebase: unstaged changes" error from pulling before committing local changes, and rebase conflicts where git couldn't automatically merge two fully-regenerated versions of the same file.

Solution: A shared concurrency group was added so the two workflows can't push to the same file at the same time. The commit order was fixed to commit local changes first, then pull with rebase, then push. And since these CSVs are fully regenerated by the scripts each time rather than hand-edited, rebase conflicts are resolved with git pull --rebase -X theirs — there's no real merge to compute, and the next run produces a correct file regardless.

4. Partial or corrupted file reads

Problem: A script reading a CSV while another process was still writing to it could catch the file mid-write.

Solution: Every CSV write across ingestion, training, and inference now uses an atomic write — the new data is written to a temporary file first, then swapped into place, so a reader never sees a half-written file.

9. Critical Evaluation / Limitations

The limitations are stated as follows:

- No automated test suite exists. Feature engineering, the atomic-write helpers, and the SHAP pipeline were all validated manually by re-running the pipeline and inspecting outputs, not by CI-enforced unit or integration tests.
- There is no input validation on ingested API data — a malformed or out-of-range reading from Open-Meteo would pass through the pipeline unchecked.
- The daily “best model” selection is based on that day's own test split, which introduces day-to-day noise into which model gets served, rather than being benchmarked against a single, stable held-out set.
- Exception handling in the retry() helper is broad (it catches bare Exception), which correctly absorbs transient network/service failures but will also silently retry a genuine bug several times before it eventually surfaces.

- Using git-committed CSV files as a shared datastore between two scheduled workflows was a deliberate trade-off to avoid standing up additional infrastructure, but it is the direct root cause of the entire class of race conditions documented in.
- There is no monitoring or alerting if a scheduled workflow silently fails — in that scenario, the dashboard would keep showing increasingly stale data with no visible warning to a user.
- The training/validation/test split used during model development covers non-overlapping seasonal windows, which limits how confidently month-based feature importance at longer horizons can be interpreted as genuinely seasonal rather than partly an artifact of the split.

10. Conclusion & Future Work

Pearls AQI Predictor delivers a working, automated, end-to-end AQI forecasting system for Karachi that meets nearly all of its specification, with its one significant deviation — the Feature Store access pattern — made deliberately, backed by concrete evidence, and documented rather than hidden. All three model families beat a naive persistence baseline at every forecast horizon, with Random Forest the most consistently strong performer, and the system's daily model-selection logic, shared feature-engineering code, and SHAP-based explainability together give it a reasonable degree of robustness and interpretability for its scale.

A few directions would meaningfully strengthen the project going forward:

- Introduce a stable, held-out benchmark set for model selection, rather than re-splitting per training run, to remove day-to-day selection noise.
- Add basic input validation on ingested API data and a lightweight automated test suite around the feature-engineering and atomic-write logic.
- Add monitoring/alerting for silent workflow failures, so stale dashboard data is surfaced rather than presented as current.
- Re-run the train/validation/test split across full, overlapping years once enough history has accumulated, to validate whether month's rising SHAP importance at longer horizons reflects genuine seasonality rather than partial split-membership leakage.
- Revisit the Feature Store access pattern periodically — if Hopsworks' Query Service reliability improves, some of the CSV-caching workaround could potentially be relaxed, though the current architecture would need no changes to keep working exactly as-is either way.
- Replace the CSV-based caching layer with a proper database such as Supabase. This would keep the same "fast, always-available cache" role the CSVs currently serve, but remove git-related issues like push races and rebase conflicts entirely, since a database handles concurrent reads/writes natively instead of relying on file-level atomic writes.
- Extend the system to multiple cities instead of just Karachi. This would test whether the current feature set and lag structure generalise to other locations, and would also make the daily training run more useful to compare across cities with different pollution profiles and seasonal patterns.
- Compare the current direct forecasting approach (a separate model trained to predict each horizon directly) against a recursive forecasting approach (a single 1-hour-ahead model applied repeatedly to generate 24h/48h/72h forecasts), to see which produces lower error and whether recursive error accumulation is a real problem at longer horizons here.

- Try a genuine ensemble (e.g. averaging or stacking Ridge, Random Forest, and the Neural Network) instead of picking one winner per horizon, to check whether combining models reduces variance in the daily forecasts compared to the current selection approach.
- Add basic model-drift monitoring — tracking how each model's test MAE changes over time — to catch a model quietly degrading before it noticeably affects the dashboard.